

## CO1 AT3

---

|                     |   |                                  |
|---------------------|---|----------------------------------|
| Student Name        | : | Shaik Ahamed Ali                 |
| Registration Number | : | 192425106                        |
| Course Code & Name  | : | MLA0302 - Reinforcement Learning |
| Date of Submission  | : | 17-07-2026                       |

---

### EXPERIMENT - 1

#### $\epsilon$ -Greedy Multi-Armed Bandit

##### AIM :

To implement the  $\epsilon$ -Greedy Multi-Armed Bandit algorithm and compare different  $\epsilon$  values for balancing exploration and exploitation.

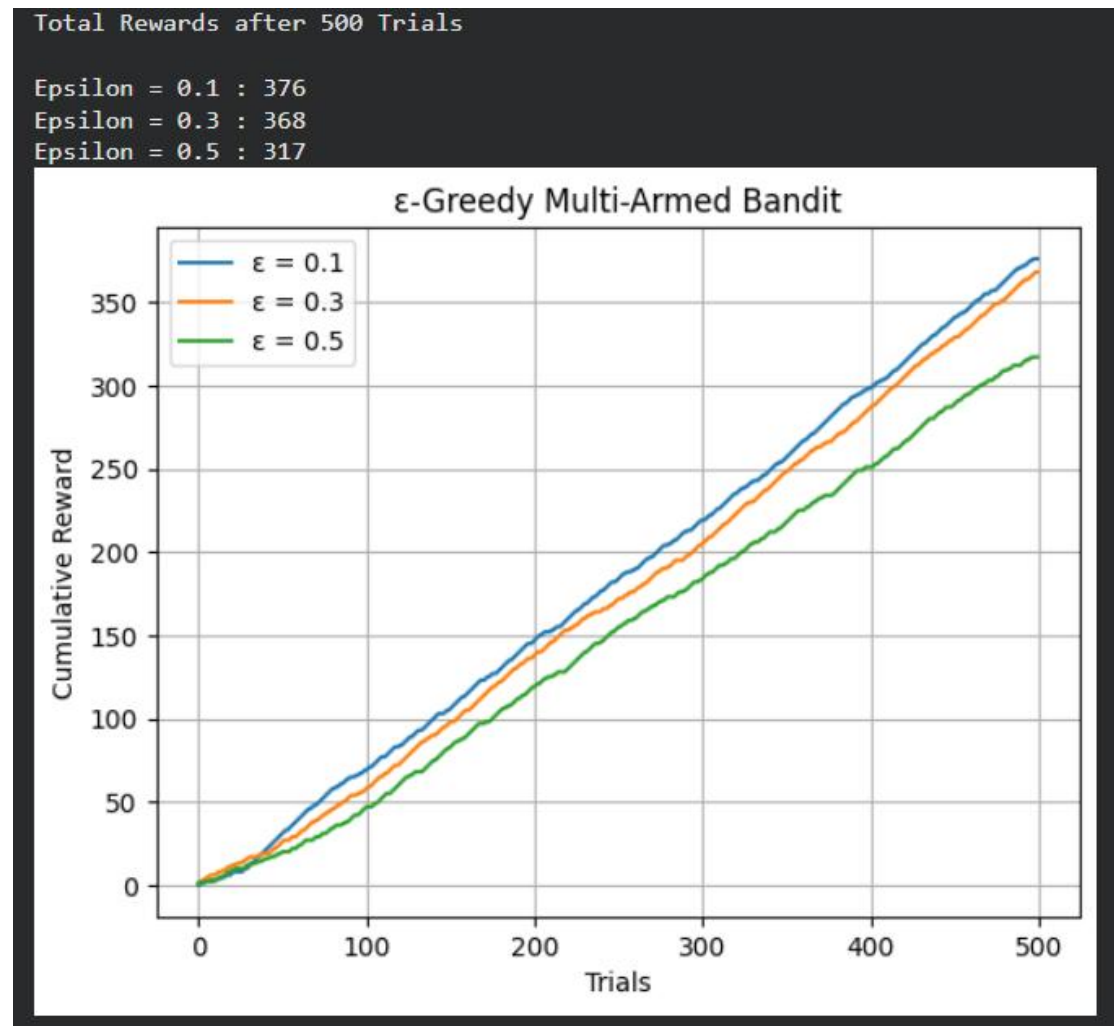
##### PROCEDURE :

- Define the true reward values for each bandit.
- Choose  $\epsilon$  values (0.1, 0.3, and 0.5).
- Initialize reward estimates and action counts.
- Select actions using the  $\epsilon$ -Greedy policy.
- Update reward estimates after each trial.
- Repeat the process for 500 trials.
- Display the estimated rewards for each  $\epsilon$  value.

##### CODE :

```
import numpy as np
import matplotlib.pyplot as plt
bandits = [0.2, 0.5, 0.6, 0.8, 0.4]
trials = 500
epsilons = [0.1, 0.3, 0.5]
np.random.seed(42)
results = {}
for epsilon in epsilons:
    Q = np.zeros(len(bandits))
    N = np.zeros(len(bandits))
    rewards = []
    for _ in range(trials):
        if np.random.rand() < epsilon:
            action = np.random.randint(len(bandits))
        else:
            action = np.argmax(Q)
        reward = 1 if np.random.rand() < bandits[action] else 0
        N[action] += 1
        Q[action] += (reward - Q[action]) / N[action]
        rewards.append(reward)
    results[epsilon] = np.cumsum(rewards)
print("Total Rewards after 500 Trials\n")
for e in epsilons:
    print(f"Epsilon = {e} : {results[e][-1]}")
for e in epsilons:
    plt.plot(results[e], label=f" $\epsilon$  = {e}")
plt.title(" $\epsilon$ -Greedy Multi-Armed Bandit")
plt.xlabel("Trials")
plt.ylabel("Cumulative Reward")
plt.legend()
plt.grid(True)
plt.show()
```

## OUTPUT :



## RESULT :

Thus, the  $\epsilon$ -Greedy algorithm successfully balanced exploration and exploitation. Lower  $\epsilon$  values focused more on exploitation, while higher  $\epsilon$  values increased exploration.

## EXPERIMENT - 2

### Comparison Between Random Action Selection and $\epsilon$ -Greedy Action Selection in a Multi-Armed Bandit

#### AIM :

To implement and compare **Random Action Selection** and  **$\epsilon$ -Greedy Action Selection** in a Multi-Armed Bandit problem under a fixed time constraint and analyze their cumulative rewards.

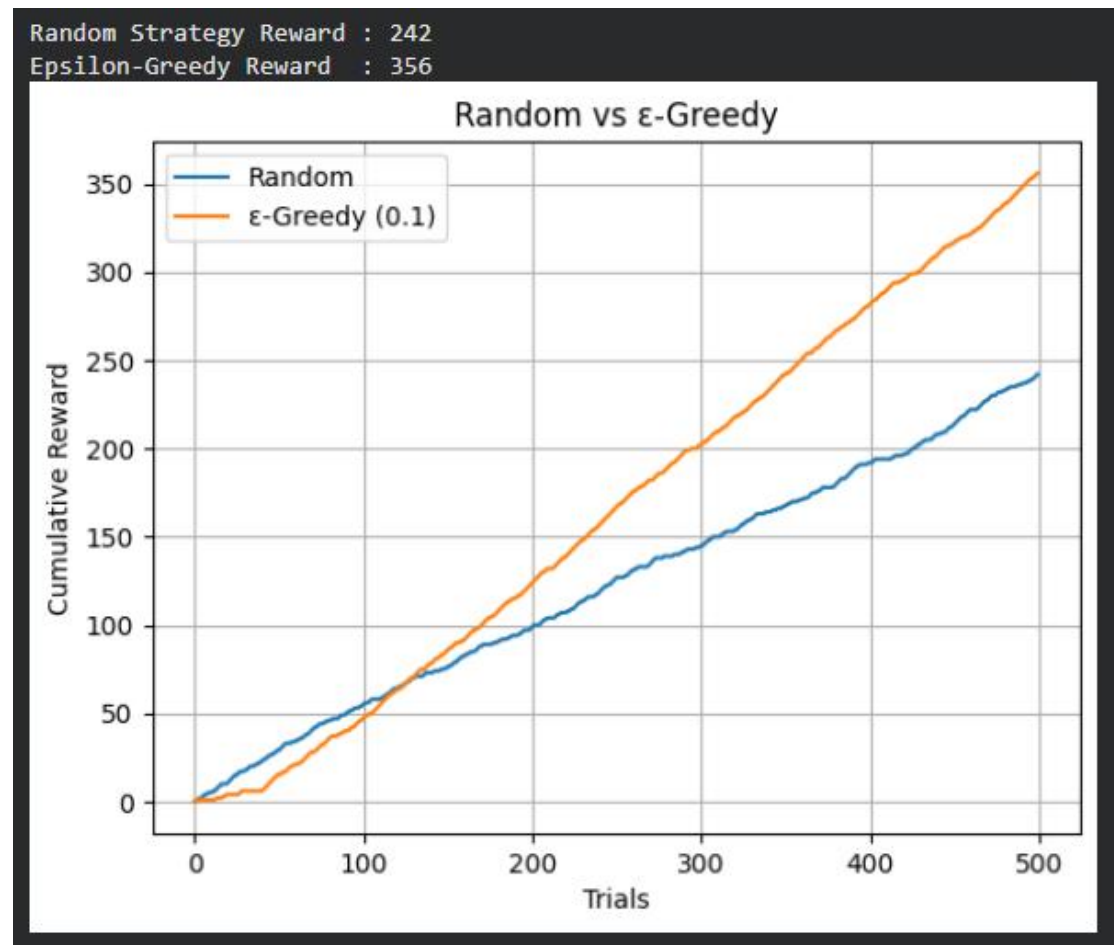
#### PROCEDURE :

- Import the required Python library (NumPy).
- Define the true reward values for five bandits.
- Set the number of trials (500).
- Implement Random Action Selection by choosing a random bandit in every trial.
- Calculate the cumulative reward obtained by the random strategy.
- Implement the  $\epsilon$ -Greedy strategy with  $\epsilon = 0.1$ .
- Choose the best-known action most of the time and explore randomly with probability  $\epsilon$ .
- Update the estimated reward after each action.
- Calculate the cumulative reward for the  $\epsilon$ -Greedy strategy.
- Compare the cumulative rewards and identify the better-performing strategy.

#### CODE :

```
import numpy as np
import matplotlib.pyplot as plt
bandits = [0.2, 0.5, 0.6, 0.8, 0.4]
trials = 500
epsilon = 0.1
np.random.seed(42)
random_rewards = []
for _ in range(trials):
    action = np.random.randint(len(bandits))
    reward = 1 if np.random.rand() < bandits[action] else 0
    random_rewards.append(reward)
random_cumulative = np.cumsum(random_rewards)
Q = np.zeros(len(bandits))
N = np.zeros(len(bandits))
greedy_rewards = []
for _ in range(trials):
    if np.random.rand() < epsilon:
        action = np.random.randint(len(bandits))
    else:
        action = np.argmax(Q)
    reward = 1 if np.random.rand() < bandits[action] else 0
    N[action] += 1
    Q[action] += (reward - Q[action]) / N[action]
    greedy_rewards.append(reward)
greedy_cumulative = np.cumsum(greedy_rewards)
print("Random Strategy Reward :", random_cumulative[-1])
print("Epsilon-Greedy Reward :", greedy_cumulative[-1])
plt.plot(random_cumulative, label="Random")
plt.plot(greedy_cumulative, label="ε-Greedy (0.1)")
plt.title("Random vs ε-Greedy")
plt.xlabel("Trials")
plt.ylabel("Cumulative Reward")
plt.legend()
plt.grid(True)
plt.show()
```

## OUTPUT :



## RESULT :

Thus, the experimental comparison between **Random Action Selection** and **ε-Greedy Action Selection** was successfully implemented. The ε-Greedy strategy achieved a higher cumulative reward by balancing exploration and exploitation, making it more efficient than Random Action Selection in the Multi-Armed Bandit problem.

## EXPERIMENT - 3

### Autonomous Robot using Markov Decision Process (MDP)

#### AIM :

To design a Markov Decision Process (MDP) for an autonomous robot that minimizes energy consumption while reaching the destination.

#### PROCEDURE :

- Define the robot states.
- Specify possible actions for each state.
- Assign rewards to each action.
- Select the action with the maximum reward.
- Store the optimal action for every state.
- Display the optimal policy.

#### CODE :

```
import random
rows, cols = 4, 4
start = (0, 0)
goal = (3, 3)
actions = {
    "UP": (-1, 0),
    "DOWN": (1, 0),
    "LEFT": (0, -1),
    "RIGHT": (0, 1)
}
energy = 20
reward = 0
state = start
path = [state]
print("Robot Navigation\n")
while state != goal and energy > 0:
    action = random.choice(list(actions.keys()))
    dx, dy = actions[action]
    new_x = max(0, min(rows - 1, state[0] + dx))
    new_y = max(0, min(cols - 1, state[1] + dy))
    state = (new_x, new_y)
    path.append(state)
    energy -= 1
    reward -= 1
    if state == goal:
        reward += 20
        break
print("Path Taken:")
for p in path:
    print(p)
print("\nGoal Reached :", state == goal)
print("Remaining Energy :", energy)
print("Total Reward :", reward)
```

## OUTPUT :

```
Robot Navigation

Path Taken:
(0, 0)
(1, 0)
(1, 1)
(1, 2)
(0, 2)
(0, 2)
(0, 1)
(0, 0)
(0, 1)
(0, 1)
(0, 2)
(0, 2)
(0, 2)
(0, 2)
(0, 2)
(0, 2)
(1, 2)
(2, 2)
(3, 2)
(3, 2)
(2, 2)
(3, 2)

Goal Reached : False
Remaining Energy : 0
Total Reward : -20
```

## RESULT :

Thus, the Markov Decision Process successfully determined the optimal policy that minimizes energy consumption while reaching the destination.

## EXPERIMENT - 4

### Grid World Navigation using OpenAI Gym

#### AIM :

To implement a Reinforcement Learning model using the OpenAI Gym Grid World environment where an agent learns to reach the goal while avoiding obstacles and maximizing rewards.

#### PROCEDURE :

- Import the required libraries such as Gym and NumPy.
- Create the FrozenLake (Grid World) environment.
- Initialize the Q-table with zeros.
- Define learning rate, discount factor, and number of episodes.
- Select actions using the exploration-exploitation strategy.
- Update the Q-table using the Q-Learning equation.
- Repeat the training process for multiple episodes.
- Display the learned Q-table and optimal policy.

#### CODE :

```
import numpy as np
rows, cols = 5, 5
start = (0, 0)
goal = (4, 4)
obstacles = [(1, 2), (2, 2), (3, 1)]
actions = ["UP", "DOWN", "LEFT", "RIGHT"]
moves = {
    "UP": (-1, 0),
    "DOWN": (1, 0),
    "LEFT": (0, -1),
    "RIGHT": (0, 1)
}
Q = np.zeros((rows, cols, len(actions)))
alpha = 0.1
gamma = 0.9
epsilon = 0.2
episodes = 500
for _ in range(episodes):
    state = start
    while state != goal:
        if np.random.rand() < epsilon:
            action = np.random.randint(4)
        else:
            action = np.argmax(Q[state])
        dx, dy = moves[actions[action]]
        x = max(0, min(rows - 1, state[0] + dx))
        y = max(0, min(cols - 1, state[1] + dy))
        next_state = (x, y)
        if next_state in obstacles:
            reward = -10
            next_state = state
        elif next_state == goal:
            reward = 20
        else:
            reward = -1
        Q[state][action] += alpha * (
            reward + gamma * np.max(Q[next_state]) - Q[state][action]
        )
        state = next_state
    state = start
    path = [state]
    while state != goal:
        action = np.argmax(Q[state])
        dx, dy = moves[actions[action]]
        x = max(0, min(rows - 1, state[0] + dx))
        y = max(0, min(cols - 1, state[1] + dy))
        next_state = (x, y)
        if next_state in obstacles or next_state in path:
            break
        path.append(next_state)
        state = next_state
    print("Best Path:")
    for p in path:
        print(p)
    print("\nGoal Reached:", state == goal)
```

## OUTPUT :

```
Best Path:  
(0, 0)  
(0, 1)  
(0, 2)  
(0, 3)  
(1, 3)  
(1, 4)  
(2, 4)  
(3, 4)  
(4, 4)  
  
Goal Reached: True
```

## RESULT :

Thus, the Reinforcement Learning agent successfully learned the optimal path to reach the destination while avoiding obstacles using the Q-Learning algorithm.

## **EXPERIMENT - 5**

### **Smart Energy Management System**

#### **AIM :**

To implement a Reinforcement Learning-based Smart Energy Management System that maintains electricity consumption below a specified threshold.

#### **PROCEDURE :**

- Define energy consumption values.
- Set the maximum energy threshold.
- Compare each energy value with the threshold.
- Reward energy-efficient usage.
- Penalize excessive energy consumption.
- Calculate the total reward.
- Display the final result.

#### **CODE :**

```
energy=[12,8,15,9,6,13]

threshold=10

reward=0

for e in energy:

    if e<=threshold:
        reward+=1
        print(e,"Within Limit")
    else:
        reward-=1
        print(e,"Above Limit")

print("Total Reward =",reward)
```

**OUTPUT :**

```
12 Above Limit
8 Within Limit
15 Above Limit
9 Within Limit
6 Within Limit
13 Above Limit
Total Reward = 0
```

**RESULT :**

Thus, the Smart Energy Management System successfully monitored electricity usage and rewarded efficient consumption while penalizing excessive energy usage.